

Kick Start 2022 - Round E

Analysis: Coloring Game

Let us use the following notation for the analysis:

- Cells painted by the bot are marked as B .
- Cells painted by John are marked as J .
- Valid cells are marked as V .
- Invalid cells are marked as X .

Let us first look at the case with $N = 5$. All cells are valid in the beginning.

Initial state: $[V, V, V, V, V]$.

The cell numbered with integer i is represented as C_i . As the bot makes the first move, it colors the leftmost valid cell, which is C_1 . This move makes the cells C_1 and C_2 invalid and reduces the number of valid cells by 2.

State after first move: $[B, X, V, V, V]$.

As John plays optimally to minimize the bot's score, he colors a cell which makes as many invalid cells as possible. Coloring the cell C_3 , which is adjacent to an invalid cell, makes the cells C_3 and C_4 invalid, while coloring the cell C_4 makes the cells C_3 , C_4 and C_5 invalid. Thus, John colors the cell C_4 , reducing the number of valid cells by 3 and leaving no more cells for the bot to color in the next turn.

State after John's move: $[B, X, X, J, X]$

Test Set 1

For $N \leq 5$, the bot can color at most 1 cell in its first move and John can make the remaining cells invalid. Thus, maximum achievable score by the bot for $N \leq 5$ is 1.

For $N = 6$, the bot can color another cell in the second move. Thus, maximum achievable score by the bot for $N = 6$ is 2.

Test Set 2

Since the bot always paints the first valid cell from the left, we can show that after each move by the bot, the number of valid cells reduces by at most 2. Also, after each move by John, the number of valid cells reduces by at most 3. We can show that this is achievable if John always paints the second valid cell from the left unless there is only one valid cell remaining. This results in reduction of the number of valid cells by at most 5 between successive turns of the bot.

In other words, the bot paints 1 cell for every 5 cells. Thus, the total number of cells the bot can paint is at least $\lceil N/5 \rceil$.

Time and Space Complexity: The answer for both the test sets can be found in $O(1)$ time and no extra space is required for calculating the answer.

Analysis: Matching Palindrome

We should not consider any string Q longer than $\mathbf{P}$ itself because $\mathbf{P}\mathbf{P}$ is a palindrome and we can always use $Q = \mathbf{P}$ if there is no shorter valid string Q . Hence, in what follows, we assume that $|Q| \leq |\mathbf{P}|$.

Since $\mathbf{P}Q$ is a palindrome, $\mathbf{P}$ must start with the reverse of Q , which is Q itself since Q is a palindrome. Thus $\mathbf{P} = QX$ for some suffix X .

Test Set 1

We can check every prefix Q of $\mathbf{P}$ and verify if Q and $\mathbf{P}Q$ are both palindromes. The shortest of such valid prefixes Q is our answer. Any given prefix Q can be verified in linear time, and the overall time complexity of this brute-force algorithm is therefore $O(\mathbf{N}^2)$.

Test Set 2

The time complexity of the naive algorithm above can be improved to $O(\mathbf{N})$ using [string hashing](#), which is a widely applicable technique for various string matching problems. In this section, though, we discuss two other approaches.

Manacher's Algorithm

So we are looking for the shortest palindromic prefix Q such that $\mathbf{P} = QX$ and QXQ is also a palindrome. It follows that the suffix X must be palindromic as well. Our task is then to split the string $\mathbf{P}$ into two palindromes Q and X such that Q is as short as possible. Finding such a split would be a trivial task if we knew in advance if any particular prefix or suffix of $\mathbf{P}$ is a palindrome.

This is where the linear time [Manacher's algorithm](#) can help. Its main purpose is to find the longest palindromic substring in a given string, however, the algorithm does more than that. Assuming $\mathbf{P} = p_1p_2p_3 \dots p_{\mathbf{N}}$, Manacher's algorithm finds the longest palindromic substring of odd length centered at any particular letter p_i and similarly the longest palindromic substring of even length centered at any particular pair of letters $p_i p_{i+1}$. Now, how do we know if a certain prefix $\mathbf{P} = p_1p_2p_3 \dots p_k$ of, say, odd length is a palindrome? It is a palindrome precisely when the length of the longest palindromic substring centered at $p_{\frac{k+1}{2}}$ is k . In a similar vein, we can test if a prefix of even length or any suffix is palindromic.

The overall time complexity of this solution is $O(\mathbf{N})$.

Optimized Brute-Force Algorithm

There is a much simpler algorithm if you trust your intuition that for the shortest prefix Q , the string $\mathbf{P}$ must be of the form $\mathbf{P} = Q^k$ for some $k \geq 1$. If this assumption holds true, then we can use a modification of the same brute-force algorithm above, where we test a prefix Q only if $|Q|$ divides $\mathbf{N}$. There are only 128 divisors of $\mathbf{N}$ in the worst case given the constraints of the problem, and that is a small enough constant.

The time complexity of this modified algorithm is $O(\mathbf{N}\sqrt{\mathbf{N}})$ or better depending on the function we use for approximating the number of divisors of $\mathbf{N}$.

It remains to show that our assumption indeed holds.

Claim: If X , Y , and $S = XY$ are all palindromic strings, then $X = Z^k$ and $Y = Z^l$ for some palindrome Z and some integers $k \geq 1$ and $l \geq 1$.

Proof: We will prove the claim using mathematical induction on $|S|$. Without loss of generality, let us assume that $|X| \leq |Y|$. Since S is a palindrome with the prefix X , S must also end in X (and so must Y). Now, since Y is a palindrome with the suffix X , Y must also start with X , and consequently S has a prefix XX . And again, since S is a palindrome, XX must be a suffix of both S and Y . And continuing this back and forth process, we may conclude that $S = X^t W X^t$, where W is a palindromic substring in the middle of S and $|W| < 2|X|$. We consider four possible cases:

- $|W| = \emptyset$: This is a base case of our inductive proof. The entire string S is a power of X , hence we can choose $Z = X$.
- $|W| = |X|$: Another base case. Since the palindrome Y equals $X^{t-1} W X^t$, W must be X , and again, S is a power of X .
- $0 < |W| < |X|$: Since $Y = X^{t-1} W X^t$ is a palindrome, the substring $W X$ in the middle must be palindromic as well. Recall that both W and X are palindromes and $|W X| < |S|$, hence, by inductive hypothesis, W and X must be powers of some palindrome Z , and so is S .
- $|X| < |W| < 2|X|$: Again, $Y = X^{t-1} W X^t$ is a palindrome, so W must be of the form $X R$, where R is palindromic. Since $|W| < |S|$, the palindromes X and R are powers of some palindrome Z by inductive hypothesis.

That concludes the proof of our claim.

To connect the dots, let us see how this claim validates our optimization of the brute-force algorithm. Suppose we split $\mathbf{P}$ into two palindromes Q and Y and $|Q|$ does not divide $\mathbf{N}$. Then, by the claim, both Q and Y are powers of a shorter palindrome Z , and Q can be disregarded in favour of Z .

Analysis: Pizza Delivery

Test Set 1

Since Ada does not have to deliver any pizzas in this test set, our task is simply to find the most profitable path of M steps starting from the location (A_r, A_c) . It is important to note that the functions $F_i(c)$ are [increasing](#) given that $K_i \geq 1$. In other words, to maximize the outcome of the functions, we should try maximizing the current number of coins c . Therefore, to find the profit of an optimal path of length M from (A_r, A_c) to a specific location (i, j) , we must find optimal paths of length $M - 1$ to all of its neighbouring locations first, apply the respective toll functions F , and take the maximum outcome. And whenever we can reduce a problem to the same but smaller sub-problems, dynamic programming can help most of the time.

Formally, let $DP_{i,j,t}$ be the maximum profit of a path of length t from (A_r, A_c) to the location (i, j) and let $L_{i,j}$ be the neighbouring locations of (i, j) ($|L_{i,j}| \leq 4$). Then

$$DP_{i,j,0} = \begin{cases} 0 & \text{if } (i, j) = (A_r, A_c) \\ -\infty & \text{otherwise} \end{cases}$$

and

$$DP_{i,j,t+1} = \max(DP_{i,j,t}, \max_{(i',j') \in L_{i,j}} DP_{i',j',t} \mathbf{OP} * \mathbf{K}_*).$$

For brevity, $\mathbf{OP} * \mathbf{K}_*$ refers to the appropriate function F when going from location (i', j') to location (i, j) . Note that Ada can choose to not move at all, hence the term $DP_{i,j,t}$ in the recurrence above.

The final answer is $\max_{1 \leq i,j \leq N} DP_{i,j,M}$. The time complexity for computing the entire DP table is $O(N^2 M)$.

Test Set 2

In the case Ada actually has to deliver some pizzas, the problem can be solved in much the same way, except that we need to keep track of the pizzas that have been delivered already. So instead of asking, what is the best way to reach the location (i, j) in t steps, we are wondering, what is the best way to deliver a subset $S \subseteq \{1, 2, \dots, P\}$ of pizzas in t steps and end up in the location (i, j) . All we have to do is extend our DP table by another dimension, namely, all the possible subsets S of pizzas.

Formally, let $DP_{i,j,S,t}$ be the maximum profit of delivering the subset of pizzas S in t steps and ending up at the location (i, j) . Then

$$DP_{i,j,S,0} = \begin{cases} 0 & \text{if } (i, j) = (A_r, A_c) \text{ and } S = \emptyset \\ -\infty & \text{otherwise} \end{cases}.$$

As for the recurrence relation, we must account for Ada's choice to deliver pizzas after each step. If none of the customers in S lives at the location (i, j) , then delivering a pizza after the next step is not an option, and our recurrence relation remains the same:

$$DP_{i,j,S,t+1} = \max(DP_{i,j,S,t}, \max_{(i',j') \in L_{i,j}} DP_{i',j',S,t} \mathbf{OP} * \mathbf{K}_*).$$

However, if, say, customer x lives at the location (i, j) and $x \in S$, then Ada can choose to deliver their pizza after making the next step, therefore, our recurrence relation becomes

$$DP_{i,j,S,t+1} = \max(DP_{i,j,S,t}, DP_{i,j,S-\{x\},t} + C_x, \max_{(i',j') \in L_{i,j}} DP_{i',j',S,t} \mathbf{OP} * \mathbf{K}_*, \max_{(i',j') \in L_{i,j}} DP_{i',j',S-\{x\},t} \mathbf{OP} * \mathbf{K}_* + C_x).$$

The final answer is $\max_{1 \leq i,j \leq N} DP_{i,j,\{1,2,\dots,P\},M}$ and the time complexity of the algorithm is now $O(N^2 2^P M)$.

Analysis: Students and Mentors

Test Set 1

In Test Set 1, you can simply try all possible options. For each student, check all the other students as a potential mentor, and choose the highest rated possible one. The time complexity of this solution will be $O(N^2)$, which is enough for this test set.

Test Set 2

In Test Set 2, we have $N \leq 10^5$, and $O(N^2)$ will take too much time, so a smarter, more efficient solution is needed. Below are two of the possible approaches.

Binary Search

If we sort an array of student ratings, then for each R_i we can find the maximum R_j such that $R_j \leq 2 \times R_i$ using binary search. The only caveat is that an extra care is required to prevent picking yourself as the highest rated possible mentor. We can prevent this from happening as follows. Make sure the binary search chooses the *rightmost* one of the several equal ratings in an array. Then if the rating of the mentor for a student i found with binary search is equal to R_i , we take rating to the left of it as an answer, or output -1 if it is the first element of an array.

The time complexity of this solution will be $O(N \log N)$ because of sorting and performing binary search N times.

Two Pointers

If we have a sorted array of student ratings, we could also use the two pointer approach to solve the problem. Let us consider students one by one, from the lowest rated ones to the highest rated ones, and try to find the ratings of the mentors for them. As we do this, we will maintain a "pointer" to the index of the highest possible rating of a mentor for the current student. Initially the pointer will be at the start of the array. Notice that as you consider the student with the higher rating and try to find the mentor for them, pointer to the highest possible mentor rating can move only to the right in the sorted array of ratings. As such, we can simply try to move the pointer to the right as far as we can for each student, and it will move no more than N times in total. An additional care is required to prevent picking yourself as a mentor, similarly to the binary search solution.

The time complexity of this solution will also be $O(N \log N)$ because you need to sort the array first. As a fun variation, it is also possible to trade time for space, and use the fact that $\max(R_i) \leq 10^6$. We can then sort the array of ratings using an array of size 10^6 and a variation of [counting sort](#), and later execute two pointer part of the solution in $O(N)$. Total time complexity of the solution will be $O(\max(N, \max(R_i)))$, or $O(N)$ as $\max(R_i) \approx 10 \times N$, although in practice it might be not much faster than $O(N \log N)$ solution.